

Final Exam Preparation: Level 3 - 4 Solutions

Reference: *LEVEL 3-4.pdf*

Below are the correct C solutions for all exercises in Level 3 and Level 4.

Note: For the linked list exercises (`ft_list_size`, `ft_list_foreach`, `ft_list_remove_if`, `sort_list`), I have included the standard 42-school `t_list` struct definition commented out at the top of the functions so you understand what structure the logic is built around.

Level 3 (Math, Algorithms, & Dynamic Allocation)

1. add_prime_sum

```
#include <unistd.h>

int ft_atoi(char *s)
{
    int res = 0;
    while (*s >= '0' && *s <= '9')
    {
        res = res * 10 + (*s - '0');
        s++;
    }
    return (res);
}

int is_prime(int n)
{
    int i = 2;
    if (n <= 1)
        return (0);
    while (i * i <= n)
    {
        if (n % i == 0)
            return (0);
        i++;
    }
}
```

```
    }
    return (1);
}

void put_nbr(int n)
{
    char c;
    if (n >= 10)
        put_nbr(n / 10);
    c = (n % 10) + '0';
    write(1, &c, 1);
}

int main(int argc, char **argv)
{
    if (argc == 2 && argv[1][0] != '-')
    {
        int n = ft_atoi(argv[1]);
        int sum = 0;

        while (n > 0)
        {
            if (is_prime(n))
                sum += n;
            n--;
        }
        put_nbr(sum);
    }
    else
        write(1, "0", 1);
    write(1, "\n", 1);
    return (0);
}
```

2. epur_str

```
#include <unistd.h>

int main(int argc, char **argv)
```

```
{
    if (argc == 2)
    {
        int i = 0;
        int flag = 0;

        while (argv[1][i] == ' ' || argv[1][i] == '\t')
            i++;

        while (argv[1][i])
        {
            if (argv[1][i] == ' ' || argv[1][i] == '\t')
                flag = 1;
            else
            {
                if (flag)
                    write(1, " ", 1);
                flag = 0;
                write(1, &argv[1][i], 1);
            }
            i++;
        }
    }
    write(1, "\n", 1);
    return (0);
}
```

3. expand_str

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 2)
    {
        int i = 0;
        int flag = 0;

        while (argv[1][i] == ' ' || argv[1][i] == '\t')
```

```
        i++;

    while (argv[1][i])
    {
        if (argv[1][i] == ' ' || argv[1][i] == '\\t')
            flag = 1;
        else
        {
            if (flag)
                write(1, "  ", 3);
            flag = 0;
            write(1, &argv[1][i], 1);
        }
        i++;
    }
    write(1, "\\n", 1);
    return (0);
}
```

4. ft_atoi_base

```
int get_val(char c)
{
    if (c >= '0' && c <= '9')
        return (c - '0');
    else if (c >= 'a' && c <= 'f')
        return (c - 'a' + 10);
    else if (c >= 'A' && c <= 'F')
        return (c - 'A' + 10);
    return (-1);
}

int ft_atoi_base(const char *str, int str_base)
{
    int i = 0;
    int sign = 1;
    int res = 0;
    int val;
```

```
    if (str[i] == '-')
    {
        sign = -1;
        i++;
    }

    while (str[i])
    {
        val = get_val(str[i]);
        if (val == -1 || val >= str_base)
            break;
        res = res * str_base + val;
        i++;
    }
    return (res * sign);
}
```

5. ft_list_size

```
#include <stdlib.h>

/*
typedef struct    s_list
{
    struct s_list *next;
    void          *data;
}                t_list;
*/

int ft_list_size(t_list *begin_list)
{
    int size = 0;

    while (begin_list)
    {
        size++;
        begin_list = begin_list->next;
    }
}
```

```
    return (size);  
}
```

6. ft_range

```
#include <stdlib.h>  
  
int *ft_range(int start, int end)  
{  
    int size;  
    int *arr;  
    int i = 0;  
  
    if (start > end)  
        size = start - end + 1;  
    else  
        size = end - start + 1;  
  
    arr = (int *)malloc(sizeof(int) * size);  
    if (!arr)  
        return (NULL);  
  
    while (i < size)  
    {  
        if (start > end)  
            arr[i] = start--;  
        else  
            arr[i] = start++;  
        i++;  
    }  
    return (arr);  
}
```

7. ft_rrange

```
#include <stdlib.h>  
  
int *ft_rrange(int start, int end)  
{
```

```
int size;
int *arr;
int i = 0;

if (start > end)
    size = start - end + 1;
else
    size = end - start + 1;

arr = (int *)malloc(sizeof(int) * size);
if (!arr)
    return (NULL);

while (i < size)
{
    if (start > end)
        arr[i] = end++;
    else
        arr[i] = end--;
    i++;
}
return (arr);
}
```

8. hidenp

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 3)
    {
        int i = 0;
        int j = 0;

        while (argv[2][j] && argv[1][i])
        {
            if (argv[2][j] == argv[1][i])
                i++;
        }
    }
}
```

```
        j++;
    }

    if (argv[1][i] == '\\0')
        write(1, "1", 1);
    else
        write(1, "0", 1);
}
write(1, "\\n", 1);
return (0);
}
```

9. lcm

```
unsigned int lcm(unsigned int a, unsigned int b)
{
    unsigned int max;

    if (a == 0 || b == 0)
        return (0);

    if (a > b)
        max = a;
    else
        max = b;

    while (1)
    {
        if (max % a == 0 && max % b == 0)
            return (max);
        max++;
    }
}
```

10. paramsum

```
#include <unistd.h>

void put_nbr(int n)
```

```
{
    char c;
    if (n >= 10)
        put_nbr(n / 10);
    c = (n % 10) + '0';
    write(1, &c, 1);
}

int main(int argc, char **argv)
{
    (void)argv;
    put_nbr(argc - 1);
    write(1, "\n", 1);
    return (0);
}
```

11. pgcd

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    if (argc == 3)
    {
        int a = atoi(argv[1]);
        int b = atoi(argv[2]);

        if (a > 0 && b > 0)
        {
            while (a != b)
            {
                if (a > b)
                    a -= b;
                else
                    b -= a;
            }
            printf("%d", a);
        }
    }
}
```

```
    }  
    printf("\n");  
    return (0);  
}
```

12. print_hex

```
#include <unistd.h>  
  
int ft_atoi(char *s)  
{  
    int res = 0;  
    while (*s >= '0' && *s <= '9')  
    {  
        res = res * 10 + (*s - '0');  
        s++;  
    }  
    return (res);  
}  
  
void print_hex(int n)  
{  
    char *hex = "0123456789abcdef";  
    if (n >= 16)  
        print_hex(n / 16);  
    write(1, &hex[n % 16], 1);  
}  
  
int main(int argc, char **argv)  
{  
    if (argc == 2)  
        print_hex(ft_atoi(argv[1]));  
    write(1, "\n", 1);  
    return (0);  
}
```

13. rstr_capitalizer

```
#include <unistd.h>

void rstr_capitalizer(char *str)
{
    int i = 0;
    while (str[i])
    {
        if (str[i] >= 'A' && str[i] <= 'Z')
            str[i] += 32;
        if ((str[i] >= 'a' && str[i] <= 'z') &&
            (str[i + 1] == ' ' || str[i + 1] == '\t' || str[i + 1] ==
'\0'))
            str[i] -= 32;
        write(1, &str[i], 1);
        i++;
    }
}

int main(int argc, char **argv)
{
    if (argc > 1)
    {
        int i = 1;
        while (i < argc)
        {
            rstr_capitalizer(argv[i]);
            write(1, "\n", 1);
            i++;
        }
    }
    else
        write(1, "\n", 1);
    return (0);
}
```

14. str_capitalizer

```
#include <unistd.h>

void str_capitalizer(char *str)
{
    int i = 0;
    while (str[i])
    {
        if (str[i] >= 'A' && str[i] <= 'Z')
            str[i] += 32;
        if ((str[i] >= 'a' && str[i] <= 'z') &&
            (i == 0 || str[i - 1] == ' ' || str[i - 1] == '\t'))
            str[i] -= 32;
        write(1, &str[i], 1);
        i++;
    }
}

int main(int argc, char **argv)
{
    if (argc > 1)
    {
        int i = 1;
        while (i < argc)
        {
            str_capitalizer(argv[i]);
            write(1, "\n", 1);
            i++;
        }
    }
    else
        write(1, "\n", 1);
    return (0);
}
```

15. tab_mult

```
#include <unistd.h>

int ft_atoi(char *s)
{
    int res = 0;
    while (*s >= '0' && *s <= '9')
    {
        res = res * 10 + (*s - '0');
        s++;
    }
    return (res);
}

void put_nbr(int n)
{
    char c;
    if (n >= 10)
        put_nbr(n / 10);
    c = (n % 10) + '0';
    write(1, &c, 1);
}

int main(int argc, char **argv)
{
    if (argc == 2)
    {
        int n = ft_atoi(argv[1]);
        int i = 1;

        while (i <= 9)
        {
            put_nbr(i);
            write(1, " x ", 3);
            put_nbr(n);
            write(1, " = ", 3);
            put_nbr(i * n);
            write(1, "\n", 1);
            i++;
        }
    }
}
```

```
    }  
}  
else  
    write(1, "\n", 1);  
return (0);  
}
```

Level 4 (Advanced Logic, Data Structures, & Linked Lists)

16. fprime

```
#include <stdio.h>  
#include <stdlib.h>  
  
int main(int argc, char **argv)  
{  
    if (argc == 2)  
    {  
        int n = atoi(argv[1]);  
        int i = 2;  
  
        if (n == 1)  
            printf("1");  
  
        while (n >= i)  
        {  
            if (n % i == 0)  
            {  
                printf("%d", i);  
                if (n == i)  
                    break;  
                printf("*");  
                n /= i;  
            }  
            else  
                i++;  
        }  
    }  
}
```

```
    printf("\n");  
    return (0);  
}
```

17. ft_itoa

```
#include <stdlib.h>  
  
int num_len(long n)  
{  
    int len = 0;  
    if (n <= 0)  
    {  
        len++;  
        n = -n;  
    }  
    while (n > 0)  
    {  
        len++;  
        n /= 10;  
    }  
    return (len);  
}  
  
char *ft_itoa(int nbr)  
{  
    long n = nbr;  
    int len = num_len(n);  
    char *res;  
  
    res = (char *)malloc(sizeof(char) * (len + 1));  
    if (!res)  
        return (NULL);  
  
    res[len] = '\0';  
  
    if (n == 0)  
        res[0] = '0';
```

```
    if (n < 0)
    {
        res[0] = '-';
        n = -n;
    }

    while (n > 0)
    {
        res[--len] = (n % 10) + '0';
        n /= 10;
    }

    return (res);
}
```

18. ft_list_foreach

```
#include <stdlib.h>

/*
typedef struct    s_list
{
    struct s_list *next;
    void          *data;
}                t_list;
*/

void ft_list_foreach(t_list *begin_list, void (*f)(void *))
{
    while (begin_list)
    {
        (*f)(begin_list->data);
        begin_list = begin_list->next;
    }
}
```

19. ft_list_remove_if

```
#include <stdlib.h>

/*
typedef struct    s_list
{
    struct s_list *next;
    void          *data;
}                t_list;
*/

void ft_list_remove_if(t_list **begin_list, void *data_ref, int (*cmp)
())
{
    t_list *curr = *begin_list;
    t_list *prev = NULL;
    t_list *tmp;

    while (curr)
    {
        if ((*cmp)(curr->data, data_ref) == 0)
        {
            if (prev == NULL)
                *begin_list = curr->next;
            else
                prev->next = curr->next;
            tmp = curr;
            curr = curr->next;
            free(tmp);
        }
        else
        {
            prev = curr;
            curr = curr->next;
        }
    }
}
```

20. ft_split

```
#include <stdlib.h>

int is_space(char c)
{
    return (c == ' ' || c == '\t' || c == '\n');
}

int word_count(char *str)
{
    int count = 0;
    while (*str)
    {
        while (*str && is_space(*str))
            str++;
        if (*str)
            count++;
        while (*str && !is_space(*str))
            str++;
    }
    return (count);
}

char *alloc_word(char *str)
{
    int len = 0;
    char *word;

    while (str[len] && !is_space(str[len]))
        len++;

    word = (char *)malloc(sizeof(char) * (len + 1));
    if (!word)
        return (NULL);

    int i = 0;
    while (i < len)
    {
        word[i] = str[i];
```

```
        i++;
    }
    word[i] = '\\0';
    return (word);
}

char **ft_split(char *str)
{
    char **arr;
    int words = word_count(str);
    int i = 0;

    arr = (char **)malloc(sizeof(char *) * (words + 1));
    if (!arr)
        return (NULL);

    while (*str)
    {
        while (*str && is_space(*str))
            str++;
        if (*str)
        {
            arr[i] = alloc_word(str);
            i++;
        }
        while (*str && !is_space(*str))
            str++;
    }
    arr[i] = NULL;
    return (arr);
}
```

21. rev_wstr

```
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
```

```
if (argc == 2)
{
    int i = 0;
    int end;
    int start;

    while (argv[1][i])
        i++;

    while (i >= 0)
    {
        while (i >= 0 && (argv[1][i] == '\0' || argv[1][i] == ' ' || argv[1][i] == '\t'))
            i--;
        end = i;
        while (i >= 0 && argv[1][i] != ' ' && argv[1][i] != '\t')
            i--;
        start = i + 1;

        int flag = start;
        while (start <= end)
        {
            write(1, &argv[1][start], 1);
            start++;
        }
        if (i >= 0)
            write(1, " ", 1);
    }
}
write(1, "\n", 1);
return (0);
}
```

22. rostring

```
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char **argv)
```

```
{
    if (argc > 1)
    {
        int i = 0;
        int first_word_start;
        int first_word_len = 0;

        while (argv[1][i] == ' ' || argv[1][i] == '\t')
            i++;

        first_word_start = i;
        while (argv[1][i] && argv[1][i] != ' ' && argv[1][i] != '\t')
        {
            first_word_len++;
            i++;
        }

        int space = 0;
        while (argv[1][i])
        {
            while (argv[1][i] == ' ' || argv[1][i] == '\t')
                i++;
            if (argv[1][i])
            {
                if (space)
                    write(1, " ", 1);
                while (argv[1][i] && argv[1][i] != ' ' && argv[1][i]
!= '\t')
                {
                    write(1, &argv[1][i], 1);
                    i++;
                }
                space = 1;
            }
        }

        if (first_word_len > 0)
        {
            if (space)
                write(1, " ", 1);
```

```
        write(1, &argv[1][first_word_start], first_word_len);
    }
}
write(1, "\n", 1);
return (0);
}
```

23. sort_int_tab

```
void sort_int_tab(int *tab, unsigned int size)
{
    unsigned int i = 0;
    int tmp;

    if (size <= 1)
        return;

    while (i < size - 1)
    {
        if (tab[i] > tab[i + 1])
        {
            tmp = tab[i];
            tab[i] = tab[i + 1];
            tab[i + 1] = tmp;
            i = 0;
        }
        else
            i++;
    }
}
```

24. sort_list

```
#include <stdlib.h>

/*
typedef struct    s_list
{
    struct s_list *next;
```

```
        void          *data;
    }                t_list;
    */

t_list *sort_list(t_list *lst, int (*cmp)(int, int))
{
    t_list *tmp = lst;
    void *swap;

    while (tmp && tmp->next)
    {
        if ((*cmp)((int)tmp->data, (int)tmp->next->data) == 0)
        {
            swap = tmp->data;
            tmp->data = tmp->next->data;
            tmp->next->data = swap;
            tmp = lst; // Reset to start
        }
        else
            tmp = tmp->next;
    }
    return (lst);
}
```